

Как задеЙствовать Module.symvers из ядра

Владислав Богданов

2026-06-26

Оглавление

1. Введение	1
2. Как этим пользоваться на практике?	1
2.1. Как они «познакомятся» при сборке?	1
2.2. Зависимость при загрузке	2
3. А теперь о ядре	2
3.1. . Где взять "Описание" функции? (Заголовочные файлы .h)	2
3.2. Где взять "Разрешение"? (Файл Module.symvers)	2
3.3. Что будет, если функции НЕТ в Module.symvers	3
3.4. Как на практике программисты ищут функции ядра?	3

1. Введение

На сколько я понимаю, ядро мы дополняем модулями, в каждом модуле есть некие функции, некоторые из этих функций мы можем вызывать снаружи.

Функцию к которой мы хотим предоставить доступ мы экспортируем:

- **EXPORT_SYMBOL(моя_функция);** — делает функцию доступной для любых модулей, даже проприетарных (закрытых).
- **EXPORT_SYMBOL_GPL(моя_функция);** — делает функцию доступной только для модулей, у которых в метаданных указана лицензия GPL (через `MODULE_LICENSE("GPL");`)

В результате в папке с получаемым драйвером мы получим файл *Module.symvers*. Это текстовый файл, в котором будет прописано:

1. **CRC (Хеш)** — контрольная сумма символа.
2. **Символ** — имя экспортируемой функции или переменной (например, `printk`, `kmalloc`).
3. **Модуль** — где этот символ определен (например, `vmlinux` для самого ядра или `snd` для звуковой подсистемы).
4. **Экспорт** — лицензионные ограничения (GPL, UNUSED и т.д.).

Что это дает?

2. Мы пишем *client.c*, где объявляем функцию как внешнюю: *extern int my_add(int a, int b);* и вызываем её.

Самое важное! Когда мы компилируем *client.ko*, система сборки (Kbuild) должна знать, где взять *Module.symvers* от *math_lib*. Если они лежат в разных папках, компилятор выдаст ошибку: *my_add undefined!*.

Чтобы это починить, при сборке *client* мы должны передать системе сборки путь к чужому *Module.symvers*. В *Makefile* это выглядит так:

```
make -C /lib/modules/$(shell uname -r)/build M=$(PWD)
KBUILD_EXTRA_SYMBOLS=/путь/к/math_lib/Module.symvers modules
```

2.2. Зависимость при загрузке

Когда скомпилируем оба модуля, то пробуем загрузить их через *insmod*:

Правильный порядок загрузки:

```
insmod math_lib.ko
insmod client.ko
```



Когда скомпилируем свой модуль, загрузчик ядра (`insmod` или `modprobe`) посмотрит в системный `Module.symvers`. Он найдет там строчку вроде:

```
0x1234abcd kmalloc vmlinux EXPORT_SYMBOL
```

И поймет: "Ага, функция `kmalloc` разрешена к использованию, ее CRC-хеш совпадает с тем, что в моем ядре. Можно грузить модуль".

3.3. Что будет, если функции НЕТ в `Module.symvers`

И так мы нашли в исходниках ядра крутую внутреннюю функцию `super_secret_kernel_func()`. Она описана в `.h` файле. Подключаем `.h` файл, пишем код, вызываем функцию. Компилятор (GCC) отработает успешно! Код скомпилируется без ошибок, потому что описание есть.

Но когда попробуем загрузить модуль (`insmod my_module.ko`), ядро выдаст ошибку: `my_module: Unknown symbol super_secret_kernel_func`. Потому что разработчики ядра не поставили макрос `EXPORT_SYMBOL` для этой функции, и ее нет в `Module.symvers`. Ядро защищает свои внутренности!

3.4. Как на практике программисты ищут функции ядра?